

PYTHON SUMMER PLAN · DAY 17

Driving the mouse, keyboard, and global hotkeys

Make Python move the cursor, type for you, and fire actions from any app with a keypress.

Wed May 27, 2026 · ~60 min · Yuvraj Ruparel

Day 16 gave you the OS bridge

You taught Python to talk to the operating system: run shell commands, launch apps, and fire AppleScript. Today you give it hands — control of the mouse and keyboard, plus hotkeys that work from anywhere.

YESTERDAY — Day 16

`subprocess.run()` to launch apps and run commands. `osascript` to send one-line AppleScript. Python could start things — but couldn't touch the screen, move the cursor, or react to a keypress while another app was in focus.

TODAY — Day 17

`pyautogui` moves and clicks the mouse, types text, and reads the screen. The `keyboard` library listens for hotkeys system-wide, so `cmd+shift+1` can fire your script no matter what app you're looking at.

pyautogui moves and clicks the cursor

What this is: pyautogui is a library that controls the physical mouse and keyboard. Calling its functions is the same as you moving the mouse by hand — every app sees a real cursor.

```
import pyautogui as pag

# screen size as (width, height) in pixels
w, h = pag.size()

# move cursor to (x, y) over 0.5 seconds
pag.moveTo(500, 300, duration=0.5)

# click where the cursor is now
pag.click()

# move to center of screen and click there
pag.click(w // 2, h // 2)
```

Coordinates

(0,0) is the top-left corner. x grows right, y grows down — not like a math graph.

duration=

Spreads the move over time so it looks human and apps register it. Skip it and the jump is instant.

// 2

Integer divide to get center pixel. Floats would error — pixels are whole numbers.

typewrite() and press() send keystrokes

What this is: These functions push keystrokes into whatever window has focus — exactly as if you typed them. Useful for auto-filling text, triggering shortcuts, or scripted form entry.

```
# type a string char-by-char, 0.05s apart
pag.typewrite("yuvraj@duke.edu", interval=0.05)

# press one key
pag.press("enter")

# hold modifiers for a shortcut (cmd+c)
pag.hotkey("command", "c")

# press+hold, do something, release
pag.keyDown("shift")
pag.press("left")
pag.keyUp("shift")
```

typewrite

Types into the focused field. interval slows it so fast apps don't drop chars.

hotkey()

Presses keys together then releases in reverse — the right way to send cmd+c, cmd+tab, etc.

keyDown/Up

Manual hold and release. Needed when a key must stay down across several actions.

The keyboard library listens system-wide

What this is: A global hotkey fires your function from any app, even when your script isn't in focus. The keyboard library runs a background listener and calls you back when the combo is pressed.

```
import keyboard

def type_email():
    pag.typewrite("yuvraj@duke.edu")

def click_center():
    w, h = pag.size()
    pag.click(w//2, h//2)

# register combos -> functions
keyboard.add_hotkey("cmd+shift+1", type_email)
keyboard.add_hotkey("cmd+shift+2", click_center)

keyboard.wait("esc") # block until Esc
```

add_hotkey

Maps a key combo to a function. Pass the function name, no () — you're handing over the function, not calling it.

wait()

Keeps the program alive so the listener keeps running. Without it the script exits instantly.

First run fails silently — grant access

What this is: On macOS Sonoma and later, the OS blocks programs from controlling the mouse/keyboard or reading the screen until you explicitly allow them. pyautogui won't error loudly — it just does nothing. This trips up everyone the first time.

Grant these before you expect anything to work

System Settings → Privacy & Security → **Accessibility**: add Terminal (or your Python IDE). Then **Screen Recording**: add it too — needed for screenshots and image search (Day 19).

PyAutoGUI failsafe — your panic button

Slam the cursor into any screen corner and pyautogui raises `FailSafeException`, killing the script. This is on by default and is your escape hatch if a runaway loop is clicking or typing out of control. Leave it on while testing. Disable only with `pag.FAILSAFE = False` once you trust the script.

The calls you'll reuse all week

Call	Library	Does	When
<code>moveTo / click</code>	pyautogui	Move cursor, press mouse	Position then act on a target
<code>typewrite / press</code>	pyautogui	Send keystrokes to focus	Auto-fill text, trigger shortcuts
<code>add_hotkey</code>	keyboard	Combo → function, global	Fire actions from any app
<code>FAILSAFE</code>	pyautogui	Corner-slam abort	Safety net during testing

TODAY'S EXERCISE · 75 MIN

Build `hotkey_demo.py`

Two global hotkeys wired to two actions. `cmd+shift+1` types your email into whatever field has focus. `cmd+shift+2` moves the cursor to the center of the screen and clicks. `Esc` quits cleanly. Grant Accessibility perms first or nothing fires.

STRUCTURE

One script, two handler functions, a listener that blocks on `keyboard.wait`.

FUNCTIONS

`type_email()` uses `typewrite`.
`click_center()` reads `size()` then clicks the midpoint.

BEHAVIOR

Hotkeys work from any app.
Test in a text field. `Esc` ends the listener loop.

Things that will bite you

Silent perm failure

No error, no movement = perms not granted. Re-check Accessibility AND restart the terminal after granting; the grant only applies to newly launched processes.

cmd vs ctrl

On mac use "command"; on Windows/Linux it's "ctrl". Hardcoding the wrong one means the hotkey just never fires. Some keyboard builds want "cmd" — verify on your machine.

Keep duration on moves

Instant jumps can be missed by apps that poll cursor position. A 0.2–0.5s duration is more reliable and easier to watch while debugging.

Test with Esc wired first

Always add `keyboard.wait("esc")` or a quit hotkey before adding click/type actions. If a loop misbehaves you want an exit that isn't the corner-slam.

Today's hands become this week's tools

DAY 18 · Autoclicker

Global hotkey toggles
clicking on/off, adjustable
CPS, hold-to-spam mode.
Hotkeys via decorator (Day 15).

DAY 19 · Screen bot

locateOnScreen() finds a
button image, then clicks it
using today's mouse calls.
Needs Screen Recording perm.

DAY 20-21 · Launcher

Hotkey-triggered routines:
open tabs, start a playlist,
fire notifications. Today's
hotkey pattern is the trigger.

The whole back half of Week 3 is built on a script that can move, type, and react to a keypress. You wrote that today.